

Capstone project report

Predicting Movie Genres from Plot Summaries

A text classifier that runs offline on a phone

Zariff

October 2026

Abstract

This project predicts a film's genres from a short plot summary. I trained the model in Python on 26,877 of the 38,397 films I kept from the CMU Movie Summary Corpus, then rewrote the prediction step in Dart so that a Flutter app can run it on a phone without a network connection. The model is simple on purpose. TF-IDF word weights feed fifteen separate logistic regressions, one per genre, and each genre has its own decision threshold chosen on validation data. On a held-out test set of 5,760 films it reaches a micro-F1 of 0.643 and a macro-F1 of 0.598, against 0.257 and 0.239 for a baseline that tags every film with every genre. Making the Dart version agree with Python was harder than building the model. Shared tokenizer test cases, a check of the exported weights against scikit-learn on all 86,400 test decisions, and a Dart test that compares the app with Python on 48 texts show that the app's probabilities match the original model's to within one millionth. The app also shows which words pushed each genre's score up. Looking at each genre's top-weighted words exposed some shortcuts the model had learned from the data. For example, two of the three strongest Family words are the names of cartoon characters.

Contents

1. Introduction	1
1.1 Related work	1
2. Data	1
2.1 The corpus	1
2.2 From 363 labels to 15 genres	2
2.3 Duplicates	2
2.4 What is left	2
2.5 Training, validation and test sets	2
3. Method	3
3.1 Tokenization	3
3.2 Features	3
3.3 One logistic regression per genre	4
3.4 Decision thresholds	5
4. Running the model on a phone	5
4.1 Exporting the weights	5
4.2 A reference implementation in plain Python	5
4.3 Checking that the phone gives the same answers	6
4.4 The app	6
5. Results	7
5.1 Overall scores	8
5.2 Results per genre	8
5.3 Effect of per-genre thresholds	10
5.4 Where it goes wrong	10
6. What the model learned	11
7. Limitations	12
8. Future work	13
9. Conclusion	14
References	15
Appendix A. Reproducing the results	16
Appendix B. Genre mapping	17

1. Introduction

Given a paragraph about a gunslinger who rides into a frontier town run by a cattle baron, most readers would say Western, and probably Action as well. I wanted to see how close a small, transparent model could get to making that call from the plot summary alone. The gunslinger plot is one of the built-in examples in my app, and the app puts Western and Action at the top (Section 4.4).

A film usually has more than one genre (2.2 on average in this data), so the task is multi-label: instead of choosing one answer, the model makes fifteen separate yes-or-no decisions. I also wanted the finished model to run inside a phone app with nothing behind it. An app like that works offline, keeps what the user types on their own device, and costs nothing to run. It also rules out any model I could not rewrite by hand in Dart, the language Flutter apps are written in. A linear model over TF-IDF word scores suits that well, because once the text has been turned into numbers, each genre needs only a weighted sum and one exponential.

I set myself four goals. The model should clearly beat a tag-everything baseline on films it has never seen, measured with macro-F1 so that rare genres count as much as common ones. Every prediction should run on the phone with no network. The app's probabilities should match the Python model's to within one millionth. And the app should show which words were behind each genre it predicts.

The rest of the report follows the data through the system. Section 2 describes the corpus and how 363 genre labels became 15. Section 3 covers the features, the model and how the thresholds were picked. Section 4 is about moving the model onto a phone and checking that the phone gives the same answers as Python. Sections 5 and 6 report the results and what the model learned, and Sections 7 and 8 cover its limitations and what I would do next.

1.1 Related work

Linear models over bags of words are a well-known baseline for text classification. Wang and Manning (2012) showed that simple linear models with word and word-pair features are hard to beat on sentiment and topic tasks, and Joulin et al. (2017) found that a linear classifier over averaged word vectors was often as accurate as deep networks while training far faster. For this exact task, Hoang (2018) predicted genres from the plot summaries of over 250,000 films, comparing Naive Bayes, Word2Vec features with XGBoost, and a recurrent network (a GRU). The GRU with a learned probability threshold did best, with an F-score of 0.56. That work used different data and a different genre list, so its scores cannot be compared directly with mine. In Hoang's best approach, a second model (an XGBoost regressor) predicts a separate threshold for each film. I use one fixed threshold per genre, chosen on validation data, and the whole model is small enough to run on a phone.

2. Data

2.1 The corpus

The data comes from the CMU Movie Summary Corpus (Bamman, O'Connor and Smith, 2013). It contains 42,306 plot summaries taken from English Wikipedia as it stood in November 2012, plus Freebase metadata for 81,741 films, including their genres. The corpus is released under the Creative Commons Attribution-ShareAlike 3.0 United States licence. Because the exported model is derived from the data, it is shared under the same licence, and the app credits the corpus on its About screen.

Two files are used. `plot_summaries.txt` holds one summary per line, prefixed by a Wikipedia ID. `movie.metadata.tsv` holds one film per line, and its ninth column is a small JSON object that maps Freebase IDs to genre names. Joining the two on the Wikipedia ID is simple apart from one detail. Nine film titles begin with a double quote, such as *“Manos”: The Hands of Fate*. A default CSV reader takes that quote as the start of a quoted field and silently drops it, so the file is read with quoting turned off and every title arrives exactly as written.

2.2 From 363 labels to 15 genres

The films that have plot summaries carry 363 different Freebase genre labels, and many of them describe how or where a film was made rather than what happens in it: “Black-and-white”, “Animation”, “World cinema”, “Short Film”, “Bollywood”, “Japanese Movies”. They are not story genres, so I did not ask the model to predict them. I kept the story genres that end up with at least 1,000 films and folded obvious sub-genres into them by hand. “Slasher” counts as Horror. “Romantic comedy” counts as both Romance and Comedy, and “Crime Thriller” as both Crime and Thriller. In total, 78 source labels feed the 15 genres; Appendix B lists them. The genres are Action, Adventure, Comedy, Crime, Drama, Family, Fantasy, Horror, Musical, Mystery, Romance, Science Fiction, Thriller, War and Western. Two more labels that sound like genres also pass the 1,000-film bar but were left out: Documentary (1,213 films) is a format rather than a kind of story, and Period piece (1,321) describes a setting.

Films whose labels map to none of the 15 genres were dropped. That removed 3,802 films. Only 411 of them have no labels at all, and the rest carry only labels such as Short Film, Documentary or Silent film. Some of these films really fit none of the 15 genres, while others were simply never given a story genre, and I could not tell which was which. Keeping them would have taught the model that even the ones missing a story genre are a “no” for every genre, so I left them all out.

2.3 Duplicates

The 42,306 summaries contain only 42,298 distinct texts, because five texts appear under more than one ID. One of them is shared by five different film versions of *Madame X*. If one copy of a summary landed in the training set and another in the test set, the model would be tested on text it had already seen, so only the first copy is kept. This catches exact copies only. Section 7 returns to the near-copies it misses.

2.4 What is left

Of the 42,306 summaries, 99 have no metadata row, 8 are repeat copies and 3,802 have none of the 15 genres. That leaves 38,397 films with an average of 2.21 genres each. A plot is 312 tokens long on average and the median is 191, so a few very long summaries pull the average up. About one plot in ten has fewer than 50 tokens. Figure 1 shows how unevenly the genres are spread. Just over half the films are dramas, while only 2.7% are westerns. This imbalance drives several later choices, in particular the class weighting in Section 3.3 and the use of macro-F1, which counts every genre equally, as the main score for tuning.

2.5 Training, validation and test sets

The films were shuffled with a fixed random seed (42) and split 70/15/15: 26,877 for training, 5,760 for validation and 5,760 for testing. The training set is used to learn the vocabulary and the model weights. The validation set is used for every decision made by comparing scores: the vocabulary size, whether to add word pairs, the class weighting, the regularisation strength and the fifteen thresholds. The test set plays no part in any of those choices and is only scored at the very end.

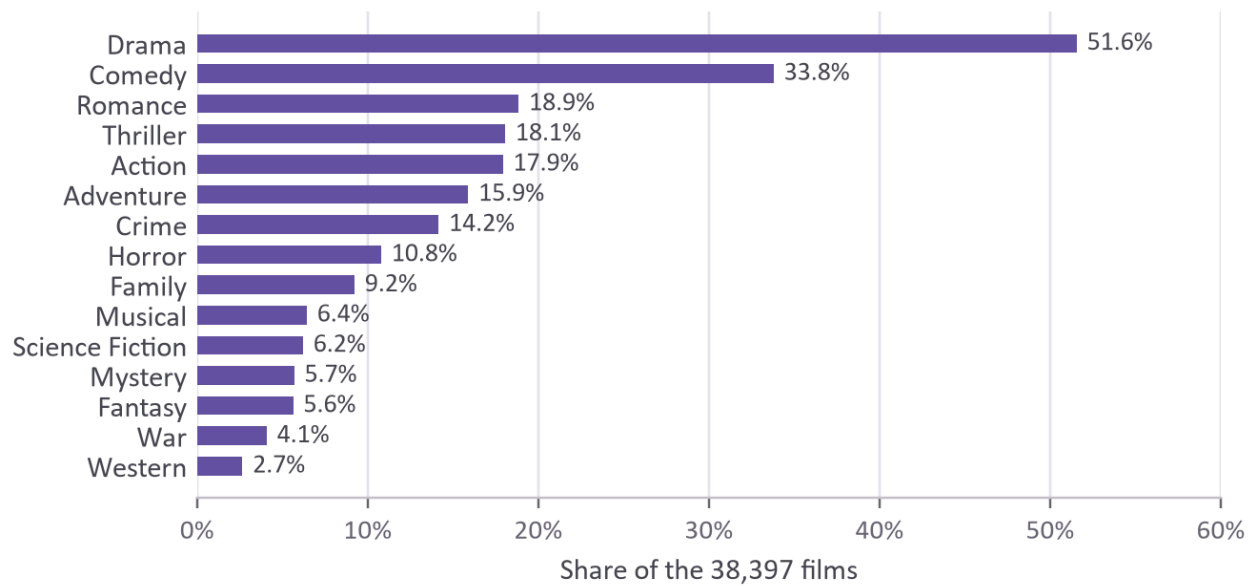


Figure 1. Share of the 38,397 films tagged with each genre. A film can have several genres, so the shares add up to more than 100%.

3. Method

3.1 Tokenization

A tokenizer decides what counts as a word. Normally that is a minor detail, but here the tokenizer exists twice, once in Python for training and once in Dart inside the app, and the two must agree exactly. If they split a sentence differently, the phone looks up different words from the ones the model learned, and its predictions drift away from the measured ones without any error message.

Python and Dart do not handle Unicode in quite the same way. Python lowercases the Kelvin sign (U+212A, which looks like a capital K) to an ordinary “k”, and it turns the Turkish capital “İ” into two characters. Dart uses its own case tables. Even the regular-expression shortcut for a “word character” differs between the two. I avoided all of this with a blunt rule: a token is a run of two or more ASCII letters, converted to lower case, and every other character separates tokens. “Spider-Man’s 2nd fight!” becomes `spider, man, nd and fight`. The cost is that accented words get split, so “café” becomes `caf`. In English-language Wikipedia plots this mostly affects names. A shared file of 19 awkward inputs, including accented letters, an emoji, the Kelvin sign, full-width letters and Windows line endings, is checked by the test suites of both languages, so if either tokenizer starts splitting any of them differently, a test fails.

3.2 Features

Each plot becomes a TF-IDF vector (Manning, Raghavan and Schütze, 2008) with one entry per vocabulary word. For a word w that appears in a plot, its value is

```
tf(w)  = 1 + ln(number of times w appears in the plot)
idf(w) = ln((1 + n) / (1 + df(w))) + 1
value  = tf(w) × idf(w)
```

where n is the number of training plots and $df(w)$ is the number of training plots that contain w . The whole vector is then divided by its length, so long and short plots are on the same scale. The logarithm in the first line gives repetition diminishing returns: a plot that says “zombie” ten times is more about zombies than one that says it once, but not ten times more. The idf term rewards rare words. “Saloon” appears in few plots and gets a high idf, while “life” appears in so many that its idf is low.

The vocabulary keeps words that appear in at least five training plots, which drops typos and one-off names, and in no more than half of them, which drops “the”, “and” and “his” without needing a stop-word list. Of the words that remain, the 20,000 most frequent are kept. The idf values are learned from the training plots only, because computing them over the test plots as well would leak information about the test set into the features.

The vocabulary size is a trade-off between accuracy and the size of the file the app has to carry. Figure 2 shows validation macro-F1 for several sizes. Keeping every eligible word (31,633 of them) beat 20,000 words by only 0.002, while making the model file about 60% larger. Adding pairs of adjacent words did not help at all and would have complicated the Dart code, so the final model uses 20,000 single words.

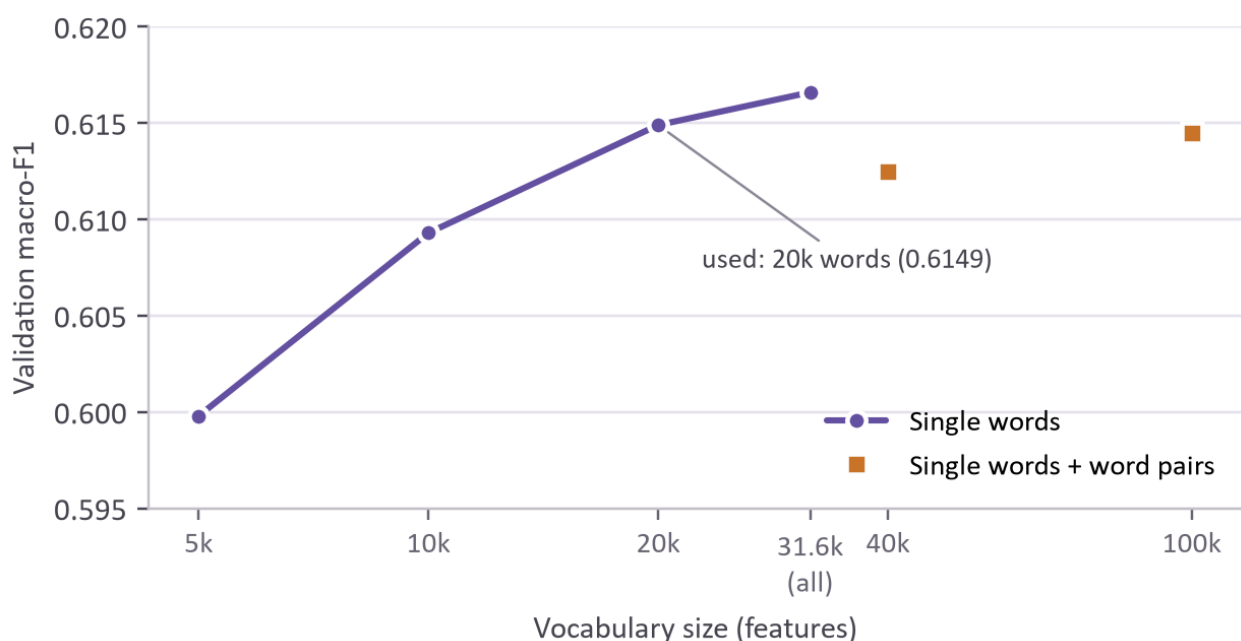


Figure 2. Validation macro-F1 for different vocabulary sizes, with and without word pairs ($C = 1$, balanced class weights).

3.3 One logistic regression per genre

Because a film can have any combination of genres, I trained fifteen independent yes-or-no classifiers, one per genre. This approach is usually called binary relevance (Tsoumakas and Katakis, 2007). For a single genre, logistic regression computes a score $z = b + \mathbf{w} \cdot \mathbf{x}$ from the TF-IDF vector $\mathbf{x}$, the learned weights $\mathbf{w}$ and an intercept b , and turns it into a probability with the sigmoid function $1 / (1 + e^{-z})$. The weights are fitted by minimising log loss with an L2 penalty, using scikit-learn’s default L-BFGS solver (Pedregosa et al., 2011). Unlike the older liblinear solver, L-BFGS leaves the intercept unpenalised.

Naive Bayes or a linear SVM would also have come down to a weighted sum on the phone. I chose logistic regression because it gives usable probabilities directly, which the per-genre thresholds and the app’s percentages need (an SVM gives only scores, and Naive Bayes probabilities tend to be extreme), and because its weights can be read as evidence for or against each genre.

The strength of the penalty is set by C , the inverse of the regularisation strength. A small C keeps the weights small, and a large C lets them fit the training data more closely. I tried six values from 0.25 to 8 and kept the one with the highest validation macro-F1. The score was 0.607 at $C = 0.25$, peaked at 0.615 at $C = 1$ and fell to 0.601 at $C = 8$, so both too much regularisation (small C) and too little (large C) cost a little.

The class imbalance needed handling. With westerns making up 2.7% of the films, a model can be 97% accurate simply by never predicting Western. I used scikit-learn's balanced class weights, which scale the classes so that a genre's few positive examples carry as much total weight as its many negative ones. Once the thresholds were tuned, the weighting made almost no difference: 0.6149 with balanced weights against 0.6151 for the best unweighted model. At a plain threshold of 0.5 and $C = 1$, though, the balanced model scored 0.590 and the unweighted one only 0.438. Balanced weights make the model much less dependent on getting the thresholds exactly right, which is why I kept them.

3.4 Decision thresholds

Logistic regression outputs a probability, but the app has to answer yes or no, and there is nothing special about 0.5. For each genre I tried every threshold from 0.05 to 0.95 in steps of 0.01 on the validation set and kept the one with the highest F1. The chosen thresholds run from 0.39 for Drama to 0.72 for War. Rare genres end up with high thresholds because balanced weighting pushes their probabilities up, and a higher threshold compensates. Drama ends up low because half of all films are dramas, so answering yes generously costs little precision and gains a lot of recall.

4. Running the model on a phone

Figure 3 shows how the pieces fit together.

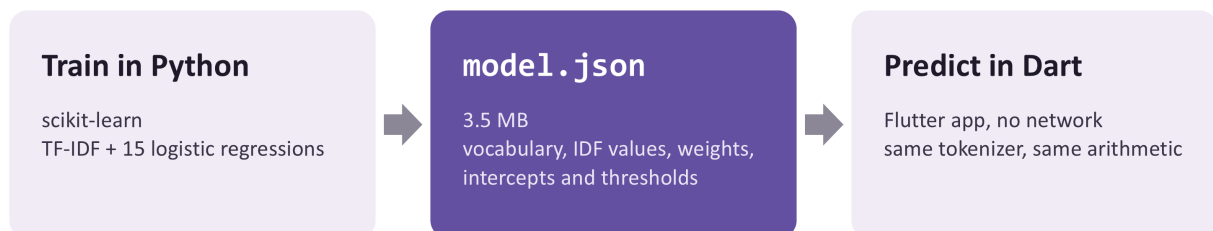


Figure 3. The model is trained in Python, exported to one JSON file, and run by the Flutter app in plain Dart.

4.1 Exporting the weights

Everything the app needs fits in one JSON file: the 20,000-word vocabulary, the 20,000 idf values, and for each genre an intercept, a threshold and 20,000 weights, which comes to 300,000 weights in all. At full precision the file is 6.6 MB. Rounding every number to seven decimal places brings it down to 3.5 MB. Rounding is only acceptable if it leaves the predictions unchanged, so the export step measures its effect (Section 4.3).

4.2 A reference implementation in plain Python

Before writing any Dart, I wrote the prediction step a second time in plain Python, without NumPy or scikit-learn, reading nothing but the JSON file. This reference version became the specification: the Dart code is a line-by-line translation of it, and the tests compare against it. Since Python 3.12 the built-in

`sum()` adds floating-point numbers with extra precision, while a loop in Dart uses ordinary addition. The difference is tiny, but to be safe the reference adds numbers in an explicit loop, in the same order as the Dart code. The sigmoid is also written in a numerically stable form, because the textbook version has to compute e^{1000} for a very negative score, and that raises an overflow error in Python.

4.3 Checking that the phone gives the same answers

Getting two languages to agree to many decimal places took more work than the model itself. In the end there are three checks, at different levels. The first is the shared tokenizer file from Section 3.1.

The second runs during training. The new model is written to a staging file, read back from disk, and used by the reference implementation to predict all 5,760 test plots. Those predictions are compared with scikit-learn's. The largest difference in any probability is 4.1×10^{-8} , and none of the 86,400 yes-or-no decisions changes. If the difference ever exceeded 10^{-6} , training would stop and leave the app's model file untouched. Only a model that passes the check replaces it.

The third is a test on the Dart side. Training also writes out 48 texts, together with Python's tokens, probabilities, decisions and top words for each one. Forty-two of the texts are real test plots, chosen so that every genre is predicted at least once, and six are awkward inputs such as an empty string, a sentence in capital letters and a French sentence containing an emoji. The Dart test runs the same texts through the app's code. Every probability must be within 10^{-6} of scikit-learn's, and every decision and top word must match exactly. To make sure this test could actually fail, I broke the Dart code on purpose by replacing $1 + \ln(\text{count})$ with the raw count. The test failed immediately, reporting a probability of 0.928 where Python expected 0.959.

4.4 The app

The Flutter app is small. It has a text box, a "Predict genres" button, a "Try an example" button that fills in one of seven plots I wrote for it, and a list of results (Figure 4). Each predicted genre gets a card with its probability, a bar marked with that genre's threshold, and up to five words that added the most to its score. The remaining genres sit in a collapsed list underneath. An About screen shows the test scores, which the app reads from the model file itself, so they always describe the model that is installed.

The model file is parsed on a background isolate, Dart's equivalent of a thread. Parsing took about 60 ms on a laptop and will take longer on a phone, and doing it in the background keeps the loading animation smooth. A prediction for a plot of average length took about 0.8 ms on the same laptop, so it simply runs when the button is pressed. The Android release build requests no internet permission at all. I installed it on a Pixel 7 emulator, switched on airplane mode and confirmed that it predicts as normal.

Showing the top words costs almost nothing because the model is linear. A genre's score is the intercept plus one term per word (the word's weight multiplied by its TF-IDF value), so each word's share of the score can be read off exactly. These shares are measured from an empty text, where the score is just the intercept. And because the TF-IDF vector is scaled to a length of one, each word's value depends on the other words in the text, so a word's share is not the same as the change you would see by deleting it. Explanation methods such as LIME (Ribeiro, Singh and Guestrin, 2016) and SHAP (Lundberg and Lee, 2017) are mainly needed for models whose scores cannot be split up like this.

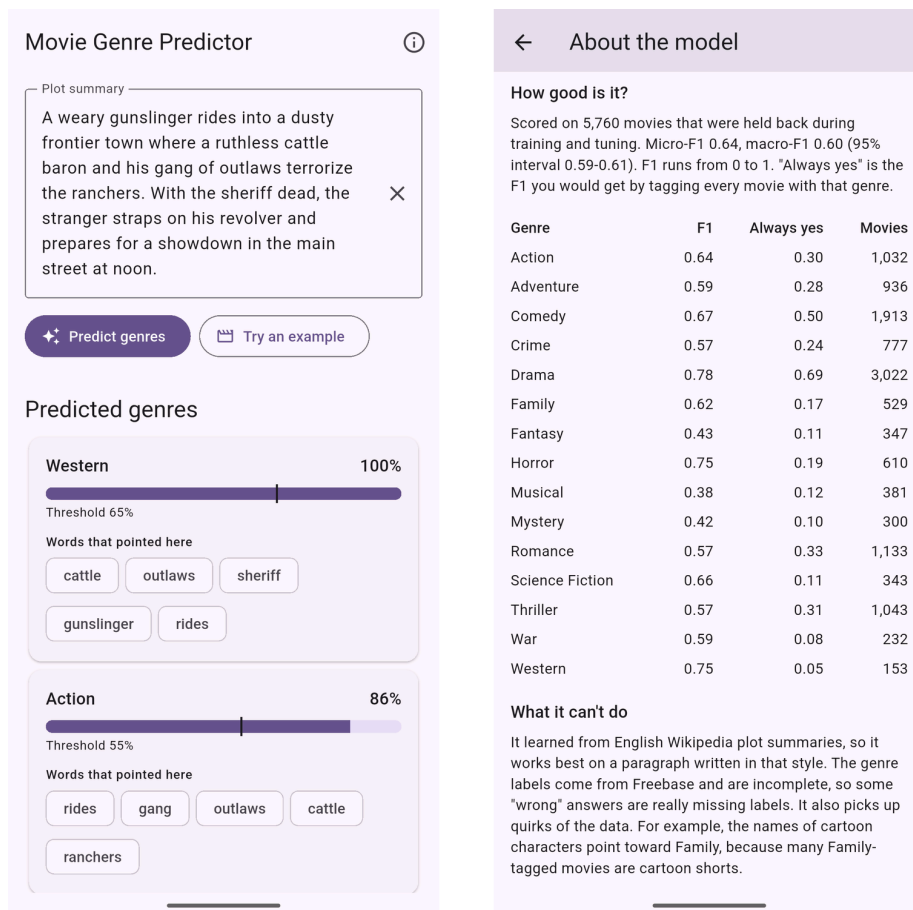


Figure 4. The release build running on an Android emulator in airplane mode. Left: the result for one of the built-in example plots. Right: the About screen with the test scores.

A code review found a bug linked to the empty-text case. With no recognised words, every genre's probability is just the sigmoid of its intercept, and for Drama that comes to 0.391, a fraction above its threshold of 0.39. So an empty text box came back as "Drama". The app now asks for a plot when the box is empty and shows no genres when none of the words are known. The model itself was left alone, so it still matches Python exactly.

In total the project has 31 Python tests and 26 Flutter tests, including the tokenizer and parity checks. The other Python tests check the TF-IDF and probability arithmetic against hand calculations, the threshold search, the genre mapping and the always-yes baseline. The Flutter widget tests drive the real screens: a western plot must produce a Western card, empty or unknown text must produce no genres, and the About screen must show the scores and the dataset credit. One of them caught an accessibility bug. The first version of the probability bar gave screen readers text where Flutter expects a number, so the bar is now hidden from them, and the card states both numbers in words instead.

5. Results

Unless marked as validation scores, the numbers in this section come from the test set.

5.1 Overall scores

Precision is the share of films given a genre that really have it, and recall is the share of films with a genre that the model found. F1 is their harmonic mean, so it is only high when both are. There are two common ways to combine F1 across the fifteen genres. Micro-F1 pools all 86,400 film-and-genre decisions before computing F1, so common genres such as Drama dominate it. Macro-F1 computes F1 for each genre and then averages, so westerns count as much as dramas. Table 1 compares the model with two simple baselines.

Table 1. Overall test results compared with two baselines.

	Micro-F1	Macro-F1
This model, with tuned thresholds	0.643	0.598
Same model, every threshold at 0.5	0.624	0.588
Tag every film with every genre	0.257	0.239

The 95% confidence intervals, from 1,000 bootstrap resamples of the test set (Efron and Tibshirani, 1993), run from 0.636 to 0.649 for micro-F1 and from 0.588 to 0.608 for macro-F1, so I would not read much into differences smaller than about 0.01. Averaged over films rather than genres, F1 is 0.631. Only 17.0% of films get exactly the right set of genres. That is a harsh measure, because with fifteen noisy labels one missing or extra genre makes the whole film count as wrong. The model also leans towards saying yes: it predicts 2.73 genres per test film, against 2.21 in the labels, which fits recall (0.718) being higher than precision (0.582). For 0.7% of test films no genre clears its threshold.

The validation macro-F1 at the chosen settings was 0.615, a little higher than the test score of 0.598. That gap is expected. C and the thresholds were picked because they did best on the validation set, and part of that advantage was luck that does not carry over to new films.

5.2 Results per genre

Table 2. Test results per genre. “Always yes” is the F1 obtained by tagging every film with the genre.

Genre	Test films	Threshold	Precision	Recall	F1	F1 at 0.5	Always yes
Action	1,032	0.55	0.577	0.721	0.641	0.634	0.304
Adventure	936	0.56	0.530	0.660	0.588	0.587	0.280
Comedy	1,913	0.48	0.604	0.741	0.666	0.665	0.499
Crime	777	0.57	0.515	0.644	0.572	0.578	0.238
Drama	3,022	0.39	0.700	0.870	0.776	0.755	0.688
Family	529	0.65	0.632	0.603	0.617	0.570	0.168
Fantasy	347	0.66	0.434	0.427	0.430	0.457	0.114
Horror	610	0.62	0.741	0.754	0.747	0.734	0.192
Musical	381	0.66	0.367	0.388	0.378	0.380	0.124
Mystery	300	0.69	0.403	0.443	0.422	0.395	0.099
Romance	1,133	0.47	0.468	0.721	0.568	0.572	0.329
Science Fiction	343	0.58	0.594	0.735	0.657	0.642	0.112

Genre	Test films	Threshold	Precision	Recall	F1	F1 at 0.5	Always yes
Thriller	1,043	0.50	0.497	0.677	0.573	0.573	0.307
War	232	0.72	0.546	0.638	0.588	0.590	0.077
Western	153	0.65	0.775	0.719	0.746	0.682	0.052
<i>Micro average</i>			0.582	0.718	0.643	0.624	0.257
<i>Macro average</i>			0.559	0.649	0.598	0.588	0.239

Figure 5 sets each genre’s F1 against the always-yes baseline, which depends only on how common the genre is: for a genre that a fraction p of films have, it equals $2p / (1 + p)$. Seen this way, Drama’s 0.776 is less impressive than it looks, because the baseline is already 0.688. Western’s 0.746 against a baseline of 0.052 is the strongest result in the table. Westerns come with a distinctive vocabulary (ranch, outlaw, cattle, saloon) and the model picks it up easily.

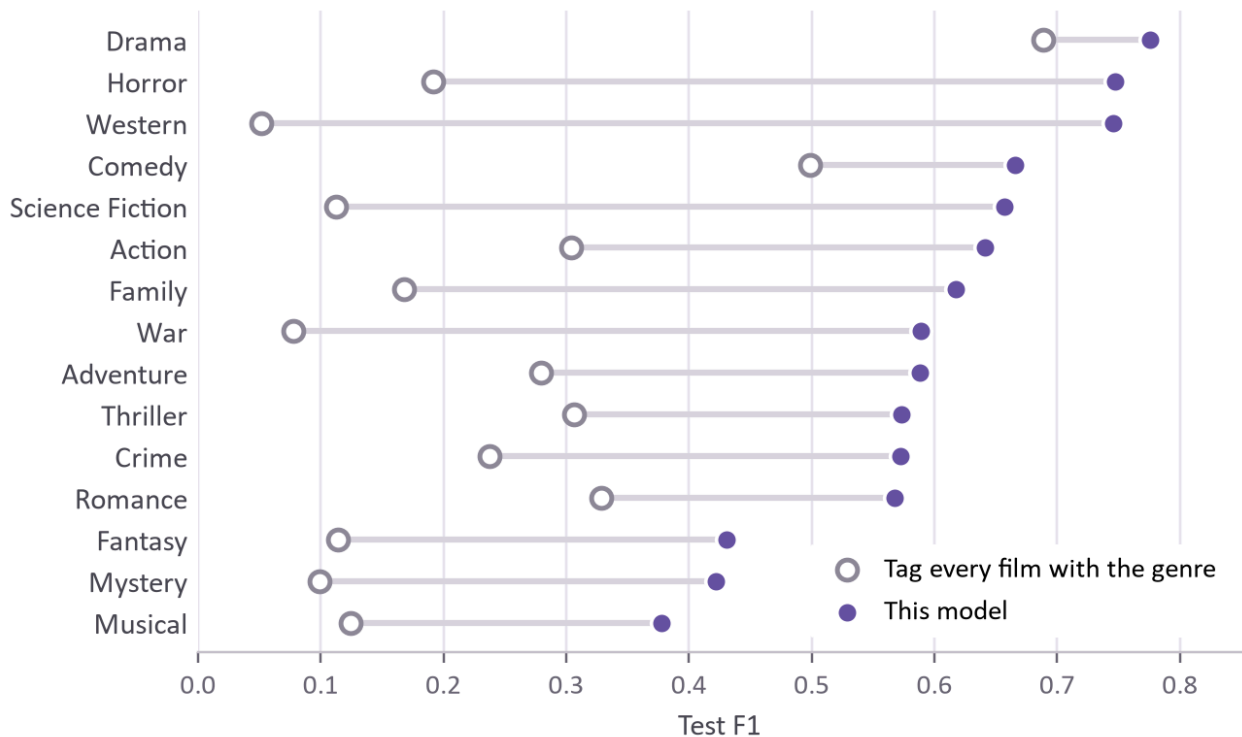


Figure 5. Test F1 per genre (filled dots) against the F1 of tagging every film with that genre (open dots).

The weakest genres are Musical (0.378), Mystery (0.422) and Fantasy (0.430). Mystery is hard to separate: 84% of mysteries are also tagged Crime or Thriller, partly because my own mapping counts Detective films as both Crime and Mystery (Appendix B). Fantasy is broad: three in four fantasy films are also tagged Family, Adventure or Comedy.

Musical is partly a labelling problem. Freebase tags 69% of the 969 Bollywood films as musicals, but only 7% of the 3,385 other films made in India. From the Bollywood films the model learns that Indian names and places go with Musical. Words about love and marriage also push towards Musical, but that link holds for films from other countries too. Together they make the model tag many other Indian films as musicals, especially love stories. The test set has 509 films made in India but not labelled Bollywood, and only 30 of them are tagged Musical. The model calls 130 of these films musicals, and 115 of those calls

are wrong, which is almost half of its 255 Musical false positives. On the 146 Bollywood test films, Musical F1 is 0.59. On the remaining 5,105 test films, which are not listed as made in India, it is only 0.36, so musicals are hard to recognise from a summary anyway.

5.3 Effect of per-genre thresholds

Tuned thresholds helped less than I expected. They raised macro-F1 from 0.588 to 0.598, and Figure 6 shows where the change came from. Nine genres improved, Western (+0.064) and Family (+0.047) most of all. Thriller's tuned threshold happened to be exactly 0.50, so nothing changed. For the other five genres the tuned threshold did slightly worse on the test set than 0.5 would have: Fantasy by 0.026, and Crime, Romance, Musical and War by 0.006 or less. A threshold chosen on 5,760 validation films is itself an estimate with noise in it, and for a small genre like Fantasy, with 347 test films, that noise is enough to cost a little.

The overall gain of 0.010 is about as large as the margin of error on the macro-F1 score in Section 5.1, but that margin is for a single score. Both versions are scored on the same films, so I also bootstrapped the difference itself. In 95% of 1,000 resamples the gain was between +0.005 and +0.016, and it was above zero in all of them, so the gain is small but consistent. Choosing the thresholds by cross-validation, so that each one rests on more films, might keep the gain while losing less on the small genres.

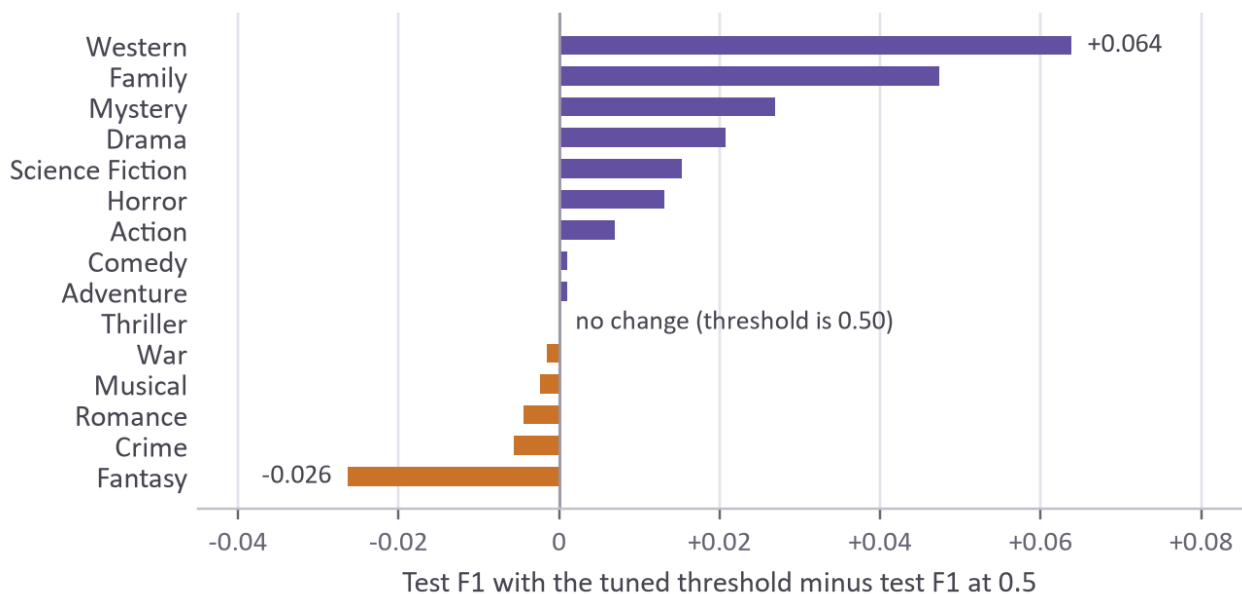


Figure 6. Change in test F1 from using the tuned threshold instead of 0.5, per genre.

5.4 Where it goes wrong

Table 3 shows four of the model's most confident false positives on the test set, with the words that pushed each one over its threshold.

I read the plots of the 25 most confident false positives, and most of them look like the first two rows. *The Life & Adventures of Santa Claus* is full of immortals and wood nymphs but is tagged only Family, and *The Double* is a CIA thriller that is not tagged Thriller. In cases like these the model is arguably right and the label is missing, which is the label noise mentioned in Section 5.1. The third row is the cartoon shortcut from Section 6 at work, although a Christmas cartoon is a reasonable Family film anyway. The last row may not be a mistake either. *The Boy Who Cried Werewolf* is tagged only Family and Comedy, but

Wikipedia calls it a comedy horror film, and Horror Comedy is one of the labels I fold into Horror (Appendix B). The confident errors mostly point at the labels, so the model's own weaknesses are easier to see in Table 2, where Musical, Fantasy and Mystery stay below 0.5 for both precision and recall.

Table 3. Four confident false positives from the test set.

Film	Its labels	Model said	Top words	What it looks like
The Life & Adventures of Santa Claus (2000)	Family	Fantasy, 0.999	christmas, magic, toys, santa, claus	Fantasy label missing
The Double (2011)	Action, Drama	Thriller, 0.993	murder, cia, fbi, investigation, agent	Thriller label missing
Bugs Bunny's Christmas Carol (1979)	Comedy	Family, 0.996	bugs, scrooge, sylvester, christmas, cat	Cartoon shortcut; Family is arguable
The Boy Who Cried Werewolf (2010)	Comedy, Family	Horror, 0.993	werewolf, blood, vampires, curse, vampire	Wikipedia calls it comedy horror

6. What the model learned

Because the model is linear, its weights can be read directly. Table 4 lists the ten words with the largest positive weight for each genre.

Table 4. The ten highest-weighted words for each genre.

Genre	Ten highest-weighted words
Action	fight, agent, kill, action, kills, martial, master, battle, rescue, revenge
Adventure	rescue, adventure, jungle, captain, yakuza, king, ranch, adventures, captured, named
Comedy	comedy, stooges, accidentally, wedding, win, date, girlfriend, eccentric, boyfriend, advice
Crime	police, murder, crime, gangster, detective, investigation, kill, robbery, murders, drug
Drama	life, relationship, returns, affair, dies, tragedy, cancer, young, emotional, prostitute
Family	dog, woody, bugs, children, kids, animals, help, christmas, cartoon, daffy
Fantasy	powers, magical, world, adventure, magic, earth, fairy, queen, king, stone
Horror	vampire, blood, creature, zombie, horror, body, ghost, strange, killer, werewolf
Musical	musical, song, music, singing, band, dance, show, sing, singer, concert
Mystery	murder, detective, killer, case, murdered, investigation, mystery, police, investigate, mysterious
Romance	love, relationship, she, romance, marry, romantic, feelings, meets, married, date
Science Fiction	alien, earth, scientist, planet, future, aliens, robot, dr, brain, machine
Thriller	murder, murdered, killing, thriller, detective, suspect, kill, police, killer, killed
War	war, german, soldiers, army, nazi, soldier, camp, japanese, captain, battle
Western	ranch, outlaw, west, cattle, cowboy, town, horse, saloon, gold, texas

Most of the lists are what you would hope to see, such as soldiers, army and nazi for War, or vampire, zombie and werewolf for Horror. Some entries, though, are artefacts of the data rather than knowledge about genres. Table 5 counts how often a few of these words appear across all 38,397 films and how strongly each one is tied to its genre.

Table 5. Some surprising top words: how many plots contain them, and what share of those plots carry the genre.

Word	Genre	Rank in genre	Plots with the word	Of those, in genre	All films in genre
woody	Family	#2	99	66%	9%
bugs	Family	#3	223	65%	9%
daffy	Family	#10	107	85%	9%
porky	Family	#11	102	82%	9%
stooges	Comedy	#2	202	99%	34%
mumbai	Musical	#13	336	31%	6%
she	Romance	#3	18,212	27%	19%
comedy	Comedy	#1	535	79%	34%

Two of the three strongest Family words, woody and bugs, are cartoon characters, and daffy and porky are not far behind. The corpus contains hundreds of Looney Tunes and Woody Woodpecker cartoons, most of them tagged Family, so the model treats the characters’ names as signs of a family film; 85% of the plots that mention Daffy are tagged Family. The Three Stooges did the same for Comedy, where 199 of the 202 plots that mention them are comedies. Mumbai is the thirteenth-strongest Musical word because many Bollywood films are tagged Musical, which means a plot set in Mumbai is nudged towards Musical whether or not anyone sings. “She” is the third-strongest Romance word. It appears in 18,212 plots and 27% of them are romances, compared with 19% of all films. My guess is that Wikipedia summaries of romances more often centre on a woman, so a romance with two male leads would get less of a boost. Finally, several genre names are among the top words for their own genre (comedy, action, horror and musical, among others), because some summaries simply say “in this comedy”. That is a mild case of the answer leaking into the input.

The app’s About screen warns users that the model picks up quirks like these, using the cartoon names as its example. Section 7 looks at what these film series do to the test score.

7. Limitations

The labels come from Freebase and are often incomplete. As Table 3 shows, a film whose summary is plainly a thriller is not always tagged Thriller, so a correct prediction can count as a false positive. The measured precision therefore understates the real precision by an amount I cannot estimate without labelling films myself. My mapping adds a little noise of its own. It misses the plain Freebase label “Crime”. Of the 26 films with plot summaries that carry it, 18 are in my data without a Crime tag.

The split is random by film, which makes some scores a little optimistic. Series of short films are spread across the training and test sets: the 202 plots that mention the Stooges split into 135 for training, 28 for validation and 39 for testing. The model gets test credit for recognising character names, and that skill would not carry over to new kinds of film. Remakes cause a smaller version of the same problem. The de-

duplication only removes identical summaries, but 8 groups of summaries share their first 30 words, mostly remakes such as the Belgian and American versions of *Lof*t. In 2 of those groups one copy is in the training set and another in the test set.

The model was trained on Wikipedia’s writing style. A one-line marketing pitch or a plot written in another language will not work well, and the app warns the user when fewer than eight words of the input are in the vocabulary. The films are also old. The median release year is 1993 and none is newer than 2014, so a plot about smartphones or social media uses words the model has rarely seen. Films whose only labels were formats such as Documentary were dropped (Section 2.2), and the 288 documentaries that were kept also carry story genres such as Drama or War. So every film the model trained on has at least one of the 15 genres. When I ran the 925 dropped documentaries through the model, it gave 894 of them at least one genre, most often Drama. The ASCII-only tokenizer splits accented words. The fifteen classifiers are independent, so the model cannot use the fact that Romance and Comedy often appear together (46% of romances are also comedies). A bag of words also ignores word order. To the model, “the police chase the gang” and “the gang chase the police” look identical. I did not compare logistic regression with other model families such as Naive Bayes or a linear SVM. Finally, I did not check calibration. A predicted 70% is not guaranteed to be right 70% of the time, and balanced class weights distort the probabilities on purpose.

Wrong genre tags do little harm, but the shortcuts from Section 6 would matter if a model like this tagged a catalogue or fed recommendations. A love story between two men gets less of the Romance boost that “she” gives, though I have not measured by how much. The Musical shortcut does show up in the results, since almost half of the Musical false positives in Section 5.2 are Indian films not labelled Bollywood. Masking character names before training would stop the model leaning on the cartoon and Stooges names. On privacy, the text never leaves the phone: the app has no networking code, and the Android release build does not have internet permission. The Dart test fixture contains plot summaries from the corpus, so, like the exported model, it is shared under CC BY-SA.

8. Future work

The first thing I would change is the split. Grouping films by series and removing near-duplicate summaries before splitting would give a stricter test score, without the small boost from series and remakes that sit on both sides of the split. After that I would look at:

- calibration curves for each genre, followed by Platt scaling (Platt, 1999) or isotonic regression if the probabilities turn out to be poorly calibrated;
- classifier chains (Read et al., 2011), which feed earlier genre predictions into later models and so can use the way genres co-occur;
- a separate C for each genre, and an L1 penalty, which sets many weights to exactly zero and so could also shrink the model file;
- storing the weights as 16-bit numbers in a binary file instead of JSON text, which would bring the model file down to about 1 MB;
- Naive Bayes and a linear SVM on the same features, to see whether logistic regression was the right choice;
- a fuller error analysis, labelling the hundred most confident false positives by hand to estimate how many are really missing labels.

9. Conclusion

The model is simple, but it did better than I expected, and it met the four goals from Section 1. Fifteen logistic regressions over 20,000 words reach a macro-F1 of 0.598 on films they have never seen, against 0.239 for tagging every film with every genre. They run inside a phone app with no network connection, and the app shows which words pushed each genre up. The exported model matches scikit-learn to within 4.1×10^{-8} on every test plot, and the Dart test holds the app to within 10^{-6} of Python. Most of the engineering effort went into making the Python and Dart code produce the same numbers, and into tests that would catch it if they ever stopped agreeing. The part I learned most from was reading each genre's top words. Seeing "daffy" and "stooges" among them showed me that the model will use any word that correlates with the labels, even a character's name. The cartoon and Stooges series also sit on both sides of my random split, which probably makes the test score a little optimistic.

References

- Bamman, D., O'Connor, B. and Smith, N. A. (2013). Learning latent personas of film characters. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics*, pages 352–361. Sofia, Bulgaria.
- Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall, New York.
- Hoang, Q. (2018). Predicting movie genres based on plot summaries. *arXiv preprint arXiv:1801.04813*.
- Joulin, A., Grave, E., Bojanowski, P. and Mikolov, T. (2017). Bag of tricks for efficient text classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics, Volume 2: Short Papers*, pages 427–431. Valencia, Spain.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, pages 4765–4774.
- Manning, C. D., Raghavan, P. and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press, Cambridge.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. Bartlett, B. Schölkopf and D. Schuurmans (eds.), *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, Cambridge, MA.
- Read, J., Pfahringer, B., Holmes, G. and Frank, E. (2011). Classifier chains for multi-label classification. *Machine Learning*, 85(3), 333–359.
- Ribeiro, M. T., Singh, S. and Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144.
- Tsoumakas, G. and Katakis, I. (2007). Multi-label classification: An overview. *International Journal of Data Warehousing and Mining*, 3(3), 1–13.
- Wang, S. and Manning, C. (2012). Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics, Volume 2: Short Papers*, pages 90–94. Jeju Island, Korea.

Appendix A. Reproducing the results

Every score, settings comparison and word list in this report can be regenerated from the repository with the commands below, using Python 3.12 and Flutter 3.44.2 or newer, run from the project folder:

```
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -r training/requirements.txt
python training/download_data.py
python training/train.py
pytest training
python training/experiments.py
python training/top_words.py
cd app
flutter test
```

Some numbers come from one-off checks that no script repeats:

- Section 2: the corpus counts and plot lengths;
- Section 4: the full-precision file size, the two timings and the deliberately broken Dart test;
- Section 5: the number of genres predicted per test film (5.1), the genre overlaps and the Indian-film breakdown for Musical (5.2), the paired bootstrap (5.3) and the error examples (5.4);
- Section 7: the Crime label, the Stooges split, the near-duplicates, the release years, the documentaries and the overlap between Romance and Comedy.

`train.py` takes about a minute and a half. It rewrites the app's model file, the Dart test fixture, and `metrics.json` and `results.md` in `training/results/`, and `experiments.py` writes `experiments.txt` there. On the same machine, running `train.py` twice produces a byte-for-byte identical model file. The project folders are `training/` (data loading, training, evaluation and export), `shared/` (the tokenizer cases both languages test), `app/` (the Flutter app, its bundled model and its tests) and `docs/` (screenshots, this report and the presentation).

Appendix B. Genre mapping

Table B1. The Freebase labels folded into each genre. A label can count towards more than one genre.

Genre	Freebase labels
Action	Action, Action/Adventure, Action Thrillers, Action Comedy, Martial Arts Film
Adventure	Adventure, Action/Adventure, Family-Oriented Adventure, Fantasy Adventure, Adventure Comedy, Swashbuckler films, Costume Adventure
Comedy	Comedy, Comedy film, Romantic comedy, Comedy-drama, Black comedy, Parody, Satire, Slapstick, Sex comedy, Screwball comedy, Crime Comedy, Comedy of manners, Horror Comedy, Fantasy Comedy, Musical comedy, Domestic Comedy, Comedy of Errors, Action Comedy, Adventure Comedy, Workplace Comedy, Political satire, Comedy Thriller, Media Satire
Crime	Crime Fiction, Crime Thriller, Crime Drama, Crime Comedy, Gangster Film, Film noir, Heist, Caper story, Detective, Detective fiction
Drama	Drama, Romantic drama, Comedy-drama, Family Drama, Crime Drama, Political drama, Melodrama, Costume drama, Historical drama, Marriage Drama, Courtroom Drama, Childhood Drama, Musical Drama
Family	Family Film, Children's/Family, Children's, Children's Fantasy, Family-Oriented Adventure
Fantasy	Fantasy, Children's Fantasy, Fantasy Comedy, Fantasy Adventure
Horror	Horror, Slasher, Zombie Film, Horror Comedy, Natural horror films, Sci-Fi Horror, Haunted House Film
Musical	Musical, Musical comedy, Musical Drama
Mystery	Mystery, Detective, Detective fiction
Romance	Romance Film, Romantic drama, Romantic comedy
Science Fiction	Science Fiction, Sci-Fi Horror, Alien Film, Time travel, Dystopia
Thriller	Thriller, Crime Thriller, Psychological thriller, Suspense, Action Thrillers, Political thriller, Erotic thriller, Comedy Thriller
War	War film, Combat Films
Western	Western